

WEEK 02 MODULE

Programming Practice

4. Sort Website Response Time (Insertion Sort)

que. 2 What is Sorting?

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- | | | |
|---------------------|---|--------|
| i. Bubble Sort | } | Common |
| ii. Selection Sort | | |
| iii. Insertion Sort | | |
| iv. Merge Sort | | |
| v. Quick Sort | | |
| vi. Heap Sort | | |

Selection Sort

Definition : Selection Sort repeatedly finds the smallest element from the unsorted portion of the array and places it at the beginning of the unsorted section.

Key Points:-

- Divides array into:
 - Sorted portion
 - Unsorted portion
- Finds the minimum element in every pass.
- Performs fewer swaps compared to Bubble Sort.
- Simple but inefficient for large datasets.

How It Works:-

Array: 5 3 8 1

Pass 1

Find smallest element:

5 3 8 1
Smallest = 1
Swap with first element:
1 3 8 5

Pass 2

Remaining unsorted:

3 8 5
Smallest is already 3.
No swap needed.
1 3 8 5

Pass 3

Remaining unsorted:

8 5
Smallest = 5
Swap:
1 3 5 8

Array sorted.

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position.

1. Problem Analysis

A web monitoring system continuously records website response times. As new response times are collected, they may not be stored in sorted order. For performance analysis and monitoring purposes, these response times need to be arranged in ascending order.

The challenge is to organize the response times efficiently so that lower and higher response values can be easily identified and analyzed.

2. Algorithm Selection Analysis

Insertion Sort is selected because it efficiently builds a sorted sequence by inserting each new element into its proper position.

Reasons for selecting Insertion Sort:

- Simple implementation.
- Builds the sorted array gradually.
- Efficient for small datasets.
- Suitable when data arrives incrementally and must be inserted into an already sorted sequence.
- Requires only a small amount of additional memory.

Unlike Bubble Sort, which repeatedly swaps adjacent elements, Insertion Sort shifts elements and inserts the current element directly into its correct location.

3. Core Idea / Logic

Insertion Sort divides the array into two parts:

- A sorted portion.
- An unsorted portion.

Initially, the first element is considered sorted.

For each subsequent element:

- Store the current element temporarily.
- Compare it with elements in the sorted portion.
- Shift larger elements one position to the right.
- Insert the current element into its correct position.

This process continues until all elements become part of the sorted portion.

4. Execution Flow

Input

- Read the number of response times (N).
- Store all response times in an array.

Processing

- Assume the first element is already sorted.
- Select the next element.
- Compare it with elements in the sorted portion.
- Shift larger elements to the right.
- Insert the selected element at its correct position.
- Repeat until all elements are sorted.

Output

- Display the response times in ascending order.

Step-by-Step Execution

Using the sample input:

5

250 120 320 180 100

Step 1

Read the number of response times.

User enters 5.

```
int n = sc.nextInt();
```

n=5

Step 2

Create an array of size n.

```
int arr[] = new int[n];
```

Step 3

Accept all response times from the user and store them in the array.

250 120 320 180 100

Array becomes:

Index : 0 1 2 3 4

Value : 250 120 320 180 100

Step 4

Assume the first element is already sorted.

Step 5

Select the next element and store it temporarily.

Step 6

Compare the selected element with elements in the sorted portion.

Step 7

Shift larger elements one position to the right.

Step 8

Insert the selected element into its correct position.

Step 9

Repeat the process for all remaining elements.

Step 10

Display the sorted response times.

5. Key Observations

- The first element is considered sorted by default.
- The sorted portion grows after every iteration.
- Elements are shifted rather than repeatedly swapped.
- Each element is inserted into its correct position within the sorted portion.
- Best-case scenario occurs when the array is already sorted.
- Worst-case scenario occurs when the array is in reverse order.
- Insertion Sort performs efficiently on small datasets.
- The algorithm sorts the array in-place without requiring an additional array.
- It is particularly useful when data arrives gradually and must be inserted into an existing sorted sequence.

6. Program Structure Analysis

i. Variable n

```
int n;
```

Purpose:

- Stores the total number of website response times.
- Determines the size of the array.
- Controls input and sorting operations

ii. Array arr[]

```
int arr[] = new int[n];
```

Purpose:

- Stores all website response times.
- Holds both unsorted and sorted values during execution.
- Provides indexed access to each response time.

iii. Input Loop

```
for(int i = 0; i < n; i++)  
{  
    arr[i] = sc.nextInt();  
}
```

Purpose:

- Accepts response times from the user.
- Stores them in the array before sorting begins.

iv. Outer Loop

```
for(int i = 1; i < n; i++)
```

Purpose:

- Selects each element from the unsorted portion.
- Starts from index 1 because the first element is already considered sorted.
- Controls the insertion process.

v. Variable key

```
int key = arr[i];
```

Purpose:

- Stores the current element that needs to be inserted into the sorted portion.
- Prevents the value from being lost while elements are shifted.

vi. Variable j

```
int j = i - 1;
```

Purpose:

- Points to the last element of the sorted portion.
- Used to compare existing elements with the key.

vii. While Loop

```
while(j >= 0 && arr[j] > key)
```

Purpose:

- Compares the key with elements in the sorted portion.
- Continues as long as larger elements exist before the correct position.
- Creates space for insertion by shifting elements.

viii. Shifting Operation

```
arr[j + 1] = arr[j];
```

Purpose:

- Moves larger elements one position to the right.
- Creates an empty location where the key can be inserted.
- Maintains the sorted order of the existing elements.

ix. Decrement Operation

```
j--;
```

Purpose:

- Moves backward through the sorted portion.
- Allows comparison with previous elements.

10. Insertion Statement

```
arr[j + 1] = key;
```

Purpose:

- Places the selected element in its correct sorted position.
- Completes one insertion step of the algorithm.

11. Output Loop

```
for(int i = 0; i < n; i++)
```

Purpose:

- Traverses the sorted array.
- Displays response times in ascending order.

Why does the loop start from i = 1?

```
for(int i = 1; i < n; i++)
```

Because Insertion Sort assumes:

Index : 0

Value : First Element is already sorted.

The algorithm begins with the second element and inserts it into the correct position relative to the sorted portion.

Example: 50 30 40 20

Initially:

[50] | 30 40 20

↑

Sorted Portion

Now 30 becomes the key and is inserted before 50.

Then:

[30 50] | 40 20

↑

Sorted Portion

The sorted portion keeps growing until the entire array is sorted.

7. Program

```
package week2_module;
```

```

import java.util.Scanner;
public class WebsiteResponseTimeInsertionSort {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input number of response times
        System.out.print("Enter the number of website response times: ");
        int n = sc.nextInt();

        // Create array
        int arr[] = new int[n];

        // Input response times
        System.out.print("Enter " + n + " response times: ");
        for(int i = 0; i < n; i++){
            arr[i] = sc.nextInt();
        }
        // Insertion Sort
        for(int i = 1; i < n; i++){
            int key = arr[i];
            int j = i - 1;
            while(j >= 0 && arr[j] > key){
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }
        // Display sorted response times
        System.out.println("Sorted Website Response Times:");
        for(int i = 0; i < n; i++){
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}

```

8. program's output

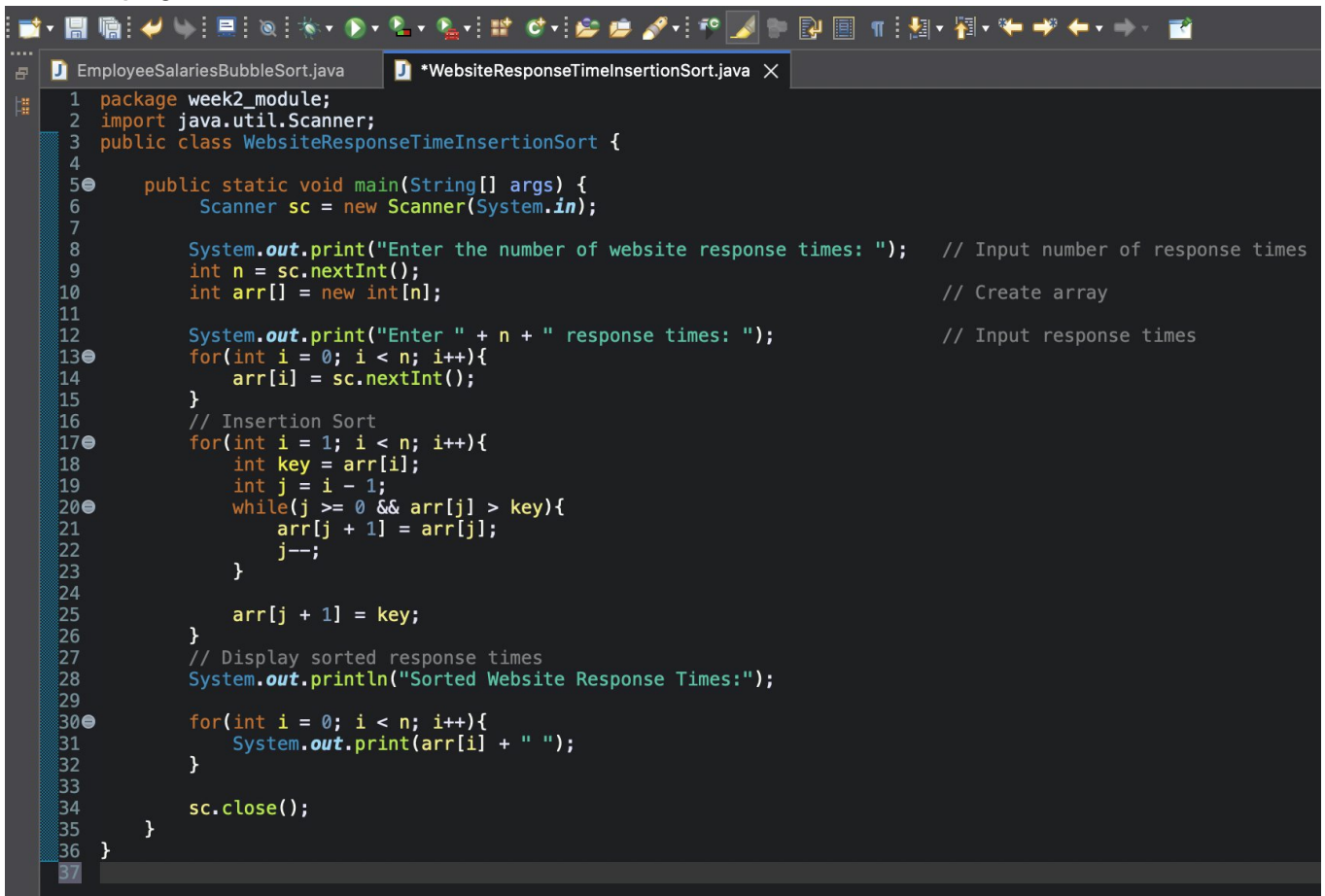
```

Enter the number of website response times: 5
Enter 5 response times: 250
120
320
180
100
Sorted Website Response Times:
100 120 180 250 320

```

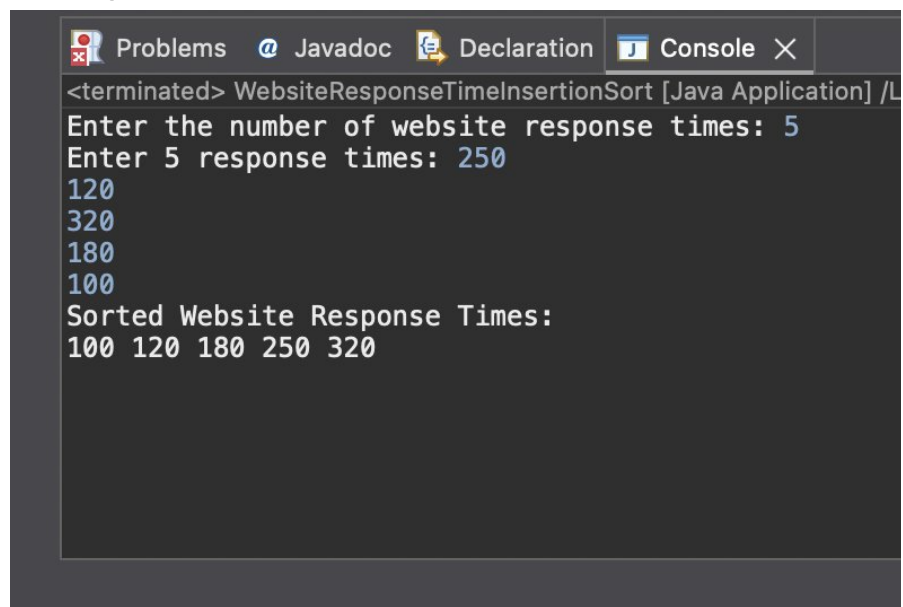

9. Screenshot

- program



```
1 package week2_module;
2 import java.util.Scanner;
3 public class WebsiteResponseTimeInsertionSort {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         System.out.print("Enter the number of website response times: "); // Input number of response times
9         int n = sc.nextInt();
10        int arr[] = new int[n]; // Create array
11
12        System.out.print("Enter " + n + " response times: "); // Input response times
13        for(int i = 0; i < n; i++){
14            arr[i] = sc.nextInt();
15        }
16        // Insertion Sort
17        for(int i = 1; i < n; i++){
18            int key = arr[i];
19            int j = i - 1;
20            while(j >= 0 && arr[j] > key){
21                arr[j + 1] = arr[j];
22                j--;
23            }
24            arr[j + 1] = key;
25        }
26        // Display sorted response times
27        System.out.println("Sorted Website Response Times:");
28
29        for(int i = 0; i < n; i++){
30            System.out.print(arr[i] + " ");
31        }
32
33        sc.close();
34    }
35 }
36 }
```

- program's output



```
<terminated> WebsiteResponseTimeInsertionSort [Java Application] /L
Enter the number of website response times: 5
Enter 5 response times: 250
120
320
180
100
Sorted Website Response Times:
100 120 180 250 320
```